



EduSmart — Arquitetura, Requisitos e Trade-offs

Grupo 8 | Rafael Camargo, Igor Lacerda, Lucca Alves, Guilherme Freitas, Heitor Rodrigues, Eduardo Brandão | 01/09/2026

1. Projeto, Domínio e Arquitetura Atual

Sobre o Projeto

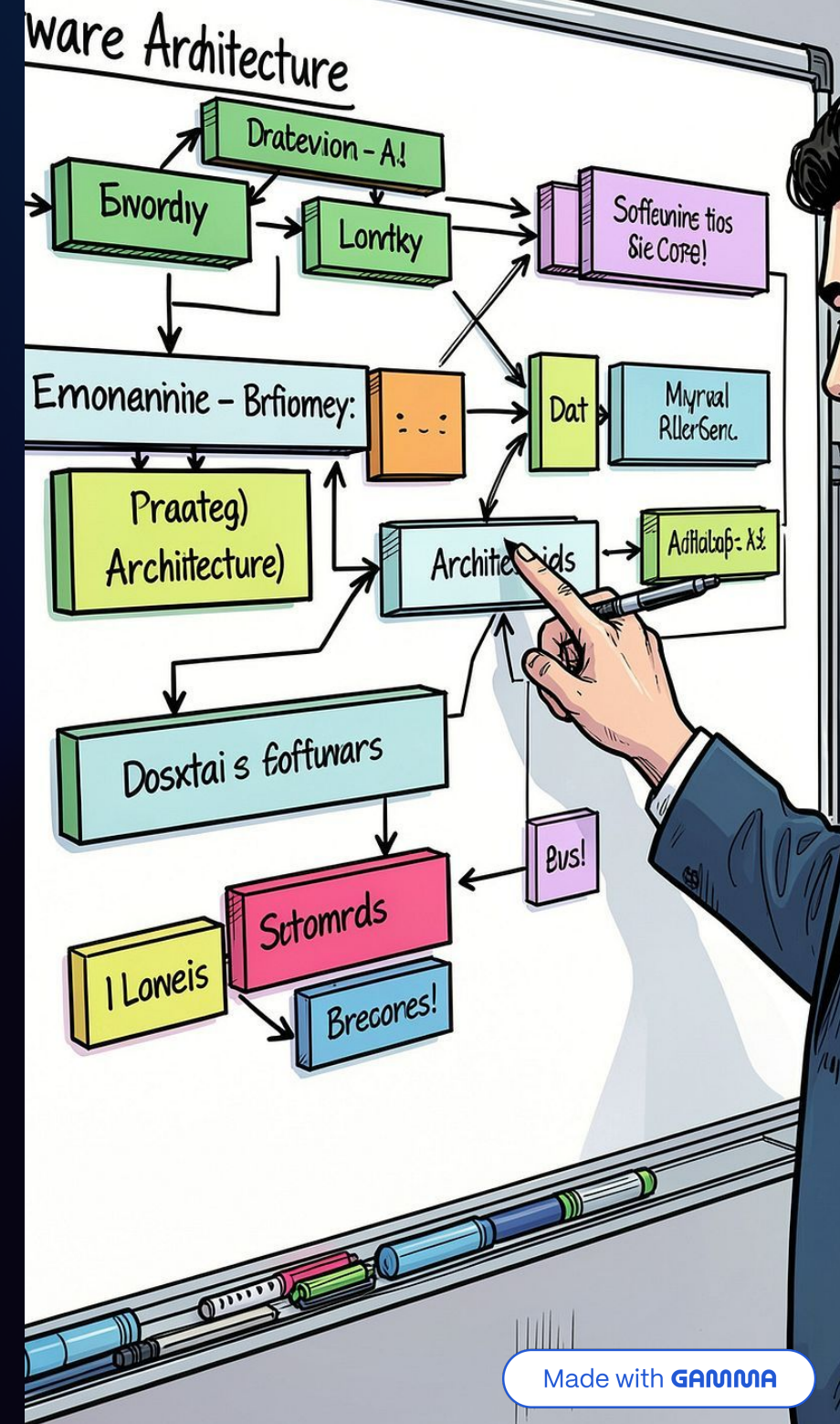
Plataforma educacional que simula controle acadêmico completo: cursos, turmas, professores, alunos, matrículas, avaliações, notas e notificações.

Arquitetura em Camadas

- **Model:** POJOs de domínio
- **Repository:** `RepositorioEmMemoria<T>` em memória
- **Service:** `ServicoEduSmart`
- Fluxo Atual

Main → `ServicoEduSmart` → `RepositorioEmMemoria` + Model + chamadas diretas às APIs legadas

⊗ **Problema crítico:** `ServicoEduSmart` concentra regra de negócio, acesso a dados e integrações externas sem abstração – acoplamento concreto direto às APIs legadas.



2. Requisitos Funcionais e Não Funcionais

RF02 — Matrícula

Não valida duplicidade, situação da turma nem capacidade máxima.

Refinamento: Impedir matrículas duplicadas e em turmas fechadas.

RF07 — Notificações

Destinatário de e-mail é fixo no código.

Refinamento: Notificar o aluno correto usando seu e-mail cadastrado.

RNF01 — Manutenibilidade

Nenhuma classe deve concentrar regra de negócio, acesso a repositório e integrações no mesmo método.

RNF02 — Consistência

0 matrículas duplicadas; 0 turmas encerradas com pendência de nota.

RNF03 — Isolamento Legado

Troca de integração não deve exigir alteração em `ServicoEduSmart`.

RNF04 — Testabilidade

Regras devem ser testáveis sem executar integrações reais.

3. Problema Arquitetural e Alternativas Analisadas

Problema Central

- Concentração de responsabilidades em `ServicoEduSmart`
- Acoplamento direto às APIs legadas sem abstração
- Falhas de validação: matrícula duplicada, turma com pendências, cálculo de média ignorando pesos

Critérios da Matriz

Desempenho · Segurança · Manutenibilidade ·
Disponibilidade/Confiabilidade · Complexidade
operacional

A — Manter Atual

Apenas correções pontuais, sem reestruturação.

Pontuação: 44

B — Ports-and-Adapters

Extrair interfaces para integrações e validação, mantendo camadas.

Pontuação: 53 ✓

C — Microserviços

Decompor em serviços independentes com comunicação por rede.

Pontuação: 29

4. Justificativa da Matriz de Decisão

Critério	A – Atual	B – Ports	C – Micro	Observação
Desempenho	8	8	4	A e B em processo único, sem latência de rede
Segurança	6	12	9	B cria ponto único de validação; A sem controle centralizado
Manutenibilidade	9	15	9	A concentra 5 responsabilidades; B reduz tamanho de ServicoEduSmart
Confiabilidade	6	6	4	C: múltiplos pontos de falha sem resiliência
Complexidade op.	15	12	3	C: múltiplos artefatos e deploy distribuído, fora do escopo
Total	44	53	29	Alternativa B vence em 3 de 5 critérios

5. Decisão: Ports-and-Adapters (Alternativa B)

Por que B?

Maior pontuação (**53 pontos**) —
melhora segurança,
manutenção e testabilidade
sem reestruturar tudo do zero.

Ações de Refatoração

- Isolar 3 integrações legadas
atrás de interfaces:
`SistemaAcademicoPort`,
`NotificacaoPort`
- Extrair validação para
classes dedicadas:
`MatriculaValidator`,
`NotaValidator`
- Manter `Model`,
`RepositorioEmMemoria<T>`
> e `Main` sem mudanças

Ganhos Esperados



RNF01 + RNF04

Regras testáveis sem APIs
reais



RNF03

Substituir integração sem
tocar `ServicoEduSmart`



Custo

Refatoração + mais
classes; exige disciplina



6. Arquitetura Proposta vs. Atual



Componentes Novos

- `MatriculaValidator`,
`NotaValidator`
- `SistemaAcademicoPort`,
`NotificacaoPort`

Componentes Refatorados

- `ServicoEduSmart` – orquestra regras, usa validadores e portas

Sem Mudanças

- `Main`, `Model`,
`RepositorioEmMemoria<T>`

Adaptadores (implementam portas)

- `ApiAcademicaLegada`,
`ApiEmailLegada`, `ApiSmsLegada`

📌 Integrações legadas isoladas – substituíveis sem alterar `ServicoEduSmart`.

7. Conclusão e Trade-off Aceito

Conclusão

A arquitetura em camadas é **adequada ao projeto**. O problema não está no estilo arquitetural, mas na **concentração de responsabilidades** e no acoplamento direto às APIs legadas dentro de `ServicoEduSmart`.

Por que descartar C (Microserviços)?

Exige infraestrutura, deploy distribuído e resiliência **incompatíveis** com o tamanho atual e o cronograma da disciplina. Pontuação 29 — o menor dentre as três alternativas.

Investimento Agora

Refatoração incremental mantendo camadas, evitando custo operacional de microserviços.

Trade-off Aceito

Proposta Final

Extrair `MatriculaValidator`, `NotaValidator`, `SistemaAcademicoPort` e `NotificacaoPort` — mantendo **simplicidade operacional** e legibilidade do projeto nesta etapa.

Evitar Amanhã

Dívida técnica que crescerá ao longo do semestre se o acoplamento permanecer.